

Modelo relacional

Normalização de dados 1NF, 2NF, 3NF, BCNF

Introdução à normalização

❶ A **normalização de uma base de dados** é um processo sistemático de organização de dados que visa eliminar as redundâncias e garantir a integridade da informação.

Imagine uma empresa que guarda os mesmos dados em vários locais diferentes. Quando algo muda, é preciso atualizar em todo o lado. Se nos esquecermos de uma dessas atualizações, surgem inconsistências que podem comprometer decisões empresariais.

O processo de normalização foi desenvolvido por Edgar F. Codd nos anos 70 e baseia-se na aplicação sequencial de regras, as chamadas "formas normais". Cada forma normal resolve problemas específicos e prepara a base de dados para a seguinte.

Problemas que a normalização resolve

Antes de mergulharmos nas regras técnicas, é fundamental compreender que problemas estamos a tentar resolver:

Redundância de dados

Quando a mesma informação aparece repetida em vários registos, ocupamos espaço desnecessário e criamos oportunidades para inconsistências. Por exemplo, se guardamos o nome completo de um departamento em cada registo de funcionário, repetimos essa informação centenas de vezes.

Anomalias de inserção

Situações em que não conseguimos adicionar dados sem ter outras informações que podem não estar disponíveis. Imagine que queremos registar um novo departamento, mas o sistema apenas permite fazê-lo se existir pelo menos um funcionário. Ficamos bloqueados.

Anomalias de atualização

Quando precisamos alterar a mesma informação em múltiplos locais. Se um departamento muda de nome e essa informação está replicada em 500 registos de funcionários, temos de fazer 500 atualizações. Se falharmos uma, criamos uma inconsistência.

Anomalias de eliminação

Quando ao apagar um registo perdemos involuntariamente outras informações valiosas. Se eliminarmos o último funcionário de um departamento e essa era a única forma de saber que o departamento existe, perdemos essa informação.

Primeira Forma Normal (1NF)

Uma tabela está na 1NF quando respeita três princípios fundamentais:

Atomicidade dos valores

Cada campo deve conter apenas um valor indivisível. Não podemos ter, por exemplo, uma célula com "João Silva, Maria Santos" no campo de contacto. Cada contacto deve estar num registo separado.

Ausência de grupos repetitivos

Não devemos ter colunas como "Telefone1", "Telefone2", "Telefone3". Esta abordagem limita artificialmente o número de valores e dificulta consultas. A solução passa por criar registos separados ou tabelas relacionadas.

Existência de chave primária

Cada registo deve ser identificável de forma única através de um ou mais campos que garantem que não existem duplicações completas.

- ❑ O cumprimento da 1NF estabelece a base estrutural mínima para uma base de dados relacional. É o primeiro passo para eliminar a redundância óbvia e garantir que podemos trabalhar com os dados de forma consistente.

Segunda Forma Normal (2NF)

- ❗ Para estar na 2NF, uma tabela deve primeiro cumprir a 1NF e, adicionalmente, eliminar as dependências parciais da chave primária. A **dependência parcial** ocorre quando um atributo não chave depende apenas de parte de uma chave primária composta (formada por múltiplos campos). Este cenário só pode existir quando a chave primária é composta.

Exemplo:

Numa tabela de "Inscrições em Cursos" com chave primária composta por (ID_Aluno, ID_Curso), o campo "Nome_Aluno" depende apenas do ID_Aluno, não da combinação completa. Esta dependência parcial viola a 2NF.

A solução passa por **decompor a tabela**, criando as estruturas separadas onde cada atributo não chave depende da totalidade da chave primária. No exemplo anterior, criaríamos uma tabela "Alunos" onde o nome depende completamente do ID_Aluno.

- 📄 A 2NF elimina as redundâncias que surgem quando informações sobre uma entidade são replicadas cada vez que essa entidade participa numa relação. Continuando o exemplo, sem 2NF, o nome de um aluno seria repetido em cada curso em que se inscrevesse.

Terceira Forma Normal (3NF)

- ❗ Uma tabela está em 3NF quando cumpre a 2NF e elimina todas as **dependências transitivas**. Uma dependência transitiva ocorre quando um atributo não chave depende de outro atributo não chave, em vez de depender diretamente da chave primária. É uma dependência "através de um intermediário".

Exemplo prático

Numa tabela de Funcionários, temos ID_Funcionario (chave primária),Codigo_Departamento e Nome_Departamento.

O Nome_Departamento depende do Codigo_Departamento, que por sua vez depende do ID_Funcionario.

Esta é uma dependência transitiva.

Solução

Criar uma tabela separada para Departamentos, onde o Nome_Departamento depende diretamente do Codigo_Departamento.

Na tabela Funcionários, mantemos apenas o Codigo_Departamento como chave estrangeira.

- 📌 A 3NF elimina as situações em que alterações num atributo não chave obrigam a múltiplas atualizações. Se um departamento mudar de nome e essa informação estiver em cada registo de funcionário, teríamos de atualizar todos. Com 3NF, atualizamos apenas na tabela Departamentos.

Forma Normal de Boyce-Codd (BCNF)

- ❗ A BCNF é uma versão mais rigorosa da 3NF, necessária em situações específicas onde a 3NF ainda permite algumas anomalias subtis.

Uma tabela está em BCNF quando, para cada dependência funcional $X \rightarrow Y$ (onde X determina Y), X é uma superchave (ou seja, pode identificar unicamente cada registo).

A diferença para a 3NF surge em cenários com **chaves candidatas sobrepostas**, quando existem múltiplas combinações possíveis de campos que poderiam servir como chave primária e essas combinações partilham alguns campos.

Exemplo:

Numa tabela de Consultoria, temos (ID_Cliente, ID_Especialista) como chave primária composta, mas também existe a regra de que cada especialista atende cada cliente sempre no mesmo tipo de projeto. Aqui, Tipo_Projeto depende do ID_Especialista (que não é, por si só, uma superchave), violando BCNF.

- ❏ A BCNF é particularmente relevante em modelos de dados complexos onde existem múltiplas relações entre as mesmas entidades. Embora menos comum que 1NF, 2NF e 3NF, resolve os problemas reais em sistemas empresariais sofisticados.

Quando é que devemos parar de normalizar?

É importante reconhecer que a normalização total nem sempre é o objetivo final. Em ambientes de produção, especialmente em data warehouses e sistemas analíticos, é comum **desnormalizar intencionalmente** para melhorar o desempenho de consultas.

A decisão de normalizar até que ponto depende de:

- Tipo de sistema (transacional vs. analítico)
- Padrões de acesso aos dados
- Requisitos de desempenho
- Complexidade que a equipa consegue gerir

❏ Para sistemas transacionais (OLTP - Online Transaction Processing), a normalização até 3NF ou BCNF é geralmente recomendada. Para sistemas analíticos (OLAP - Online Analytical Processing), as estruturas desnormalizadas podem ser preferíveis.

Processo prático de normalização

O processo de normalização segue uma sequência lógica:

1 Identificar todas as dependências funcionais

Compreender que os campos determinam outros campos

3 Aplicar a 2NF

Eliminar as dependências parciais de chaves compostas

5 Avaliar a BCNF

Verificar se existem violações subtis em cenários com múltiplas chaves candidatas

2 Aplicar a 1NF

Garantir a atomicidade e eliminar os grupos repetitivos

4 Aplicar a 3NF

Eliminar as dependências transitivas

6 Validar o modelo

Garantir que todas as relações estão corretas e que não se perdeu informação

Em cada etapa, é fundamental documentar as decisões tomadas e verificar que o modelo resultante preserva toda a informação original, sem introduzir as dependências artificiais.

ESTUDO DE CASO

Sistema de gestão de formações empresariais

Contexto

A empresa **FormaPro** oferece formações profissionais e precisa de uma base de dados para gerir a sua operação. Atualmente, mantém toda a informação numa única folha de cálculo com a seguinte estrutura.

Tabela Registos_Formacao

ID_Registo	Nome_Formando	Email_Formando	Telefones_Formando	Cursos_Inscritos	Formador_Curso1	Email_Formador1	Dep_Formador1	Local_Depto1
1	Ana Costa	ana@email.pt	912345678, 213456789	SQL Avançado, Python Básico	Carlos Mendes	carlos@formapro.pt	IT	Lisboa
2	Bruno Silva	bruno@email.pt	918765432	SQL Avançado	Carlos Mendes	carlos@formapro.pt	IT	Lisboa
3	Ana Costa	ana@email.pt	912345678, 213456789	Excel Empresarial	Diana Rodrigues	diana@formapro.pt	Gestão	Porto

Problemas identificados pelo diretor da FormaPro

- Quando um formando muda de número de telefone, têm de procurar todos os seus registos
- O email do formador Carlos foi alterado, mas esqueceram-se de atualizar num registo
- Querem adicionar um novo formador ao sistema, mas só podem fazê-lo se houver um curso associado
- Não conseguem facilmente listar todos os formadores do departamento de IT

❏ **Objetivo:** Normalizar esta estrutura até à 3NF (e avaliar se BCNF é necessária), documentando cada passo e explicando as decisões tomadas.

PASSO 1

Análise da situação inicial

Antes de começar a normalizar, vamos identificar os problemas específicos:

Violações evidentes:

- Campo "Telefones_Formando" contém múltiplos valores (viola atomicidade)
- Campo "Cursos_Inscritos" contém múltiplos valores (viola atomicidade)
- Informação do formando repete-se em cada curso (redundância)
- Informação do formador repete-se para cada formando (redundância)
- Informação do departamento repete-se com cada formador (dependência transitiva)

Dependências funcionais identificadas:

- ID_Registo → Todos os campos (chave primária atual)
- Email_Formando → Nome_Formando
- Email_Formador → Dep_Formador, Local_Dep
- Dep_Formador → Local_Dep

PASSO 2

Aplicação da 1NF

Objetivo: Eliminar valores múltiplos e garantir atomicidade.

Decisão 1: Separar telefones em registos individuais

Como um formando pode ter múltiplos telefones e esse número não é fixo, criamos uma tabela separada para telefones em vez de criar múltiplas colunas.

Decisão 2: Separar cursos em registos individuais

Cada inscrição de um formando num curso será um registo separado, permitindo modelar adequadamente a relação muitos para muitos entre formandos e cursos.

Resultado após 1NF:

Tabela Formandos_Temp

ID_Formando	Nome_Formando	Email_Formando
1	Ana Costa	ana@email.pt
2	Bruno Silva	bruno@email.pt

Tabela Telefones_Formandos_Temp

ID_Formando	Telefone
1	912345678
1	213456789
2	918765432

Tabela Inscricoes_Temp

ID_Inscricao	ID_Formando	Nome_Curso	Nome_Formador	Email_Formador	Dep_Formador	Local_Dep
1	1	SQL Avançado	Carlos Mendes	carlos@formapro.pt	IT	Lisboa
2	1	Python Básico	Outro Formador	outro@formapro.pt	IT	Lisboa
3	2	SQL Avançado	Carlos Mendes	carlos@formapro.pt	IT	Lisboa
4	1	Excel Empresarial	Diana Rodrigues	diana@formapro.pt	Gestão	Porto

Nota: Estamos em 1NF porque todos os campos contêm valores atómicos e cada linha é identificável pela chave primária.

PASSO 3

Aplicação da 2NF

Análise de dependências parciais

Na tabela **Inscricoes_Temp**, precisamos identificar se existem campos que dependem apenas de parte da chave primária.

Se usássemos (ID_Formando, Nome_Curso) como chave primária composta:

- Nome_Formador depende apenas de Nome_Curso (não do formando)
- Email_Formador depende apenas de Nome_Curso
- Dep_Formador depende apenas de Nome_Curso
- Local_Dep depende apenas de Nome_Curso

Isto são dependências parciais! Precisamos de separar estas informações.

☐ **Decisão:** Criar tabelas separadas para Cursos e Formadores, eliminando as dependências parciais.

Resultado após 2NF:

Tabela Formandos

ID_Formando	Nome_Formando	Email_Formando
1	Ana Costa	ana@email.pt
2	Bruno Silva	bruno@email.pt

Tabela Telefones_Formandos

ID_Formando	Telefone
1	912345678
1	213456789
2	918765432

Tabela Formadores_Temp

ID_Formador	Nome_Formador	Email_Formador	Dep_Formador	Local_Dep
1	Carlos Mendes	carlos@formapro.pt	IT	Lisboa
2	Outro Formador	outro@formapro.pt	IT	Lisboa
3	Diana Rodrigues	diana@formapro.pt	Gestão	Porto

Tabela Cursos

ID_Curso	Nome_Curso	ID_Formador
1	SQL Avançado	1
2	Python Básico	2
3	Excel Empresarial	3

Tabela Inscricoes

ID_Inscricao	ID_Formando	ID_Curso
1	1	1
2	1	2
3	2	1
4	1	3

☐ **Raciocínio:** Agora, se o email de um formador mudar, alteramos apenas na tabela Formadores. Se um curso mudar de formador, alteramos apenas na tabela Cursos. A redundância foi significativamente reduzida.

PASSO 4

Aplicação da 3NF

Análise de dependências transitivas

Na tabela **Formadores_Temp**, identificamos:

- ID_Formador → Dep_Formador (dependência direta da chave)
- Dep_Formador → Local_Dep (dependência entre não chaves)
- Portanto: ID_Formador → Dep_Formador → Local_Dep (dependência transitiva!)

⚠ Problema: Se o departamento de IT mudar de Lisboa para Coimbra, teríamos de atualizar todos os formadores desse departamento. Além disso, se eliminarmos o último formador de um departamento, perdemos a informação sobre a localização desse departamento.

📌 Decisão: Criar uma tabela separada para Departamentos.

Resultado final após 3NF:

Tabela Formandos

ID_Formando	Nome_Formando	Email_Formando
1	Ana Costa	ana@email.pt
2	Bruno Silva	bruno@email.pt

Tabela Telefones_Formandos

ID_Formando	Telefone
1	912345678
1	213456789
2	918765432

Tabela Formadores

ID_Formador	Nome_Formador	Email_Formador	ID_Departamento
1	Carlos Mendes	carlos@formapro.pt	1
2	Outro Formador	outro@formapro.pt	1
3	Diana Rodrigues	diana@formapro.pt	2

Tabela Departamentos

ID_Departamento	Nome_Departamento	Localizacao
1	IT	Lisboa
2	Gestão	Porto

Tabela Cursos

ID_Curso	Nome_Curso	ID_Formador
1	SQL Avançado	1
2	Python Básico	2
3	Excel Empresarial	3

Tabela Inscricoes

ID_Inscricao	ID_Formando	ID_Curso
1	1	1
2	1	2
3	2	1
4	1	3

PASSO 5

Avaliação de BCNF

📌 **Análise:** Precisamos verificar se todas as dependências funcionais têm como determinante uma superchave.

Revisão de cada tabela

Formandos

ID_Formando é a única chave candidata e determina todos os outros campos ✓

Telefones_Formandos

(ID_Formando, Telefone) é chave primária composta e é a única superchave ✓

Departamentos

ID_Departamento é a única chave candidata ✓

Formadores

ID_Formador é a única chave candidata ✓

Cursos

ID_Curso é a única chave candidata ✓

Inscricoes

(ID_Formando, ID_Curso) poderia servir de chave, mas usamos ID_Inscricao por simplicidade, não existem outras dependências funcionais problemáticas ✓

📌 **Conclusão:** O modelo está em BCNF. Não existem situações onde um determinante de dependência funcional não seja uma superchave.

Principais aprendizagens

1. Metodologia sistemática

A normalização não é um processo aleatório. Seguimos uma sequência lógica (1NF → 2NF → 3NF → BCNF) onde cada passo resolve problemas específicos.

2. Identificação de dependências

O sucesso da normalização depende da capacidade de identificar corretamente as dependências funcionais entre os dados. Isto requer compreensão profunda das regras de negócio.

3. Equilíbrio entre teoria e prática

Embora a teoria seja importante, as decisões práticas (como usar ID_Inscricao em vez de chave composta) devem considerar a usabilidade e manutenibilidade do sistema.

4. Eliminação progressiva de anomalias

- **Antes:** Mudança no email de um formador exigia múltiplas atualizações
- **Depois:** Uma única atualização na tabela Formadores
- **Antes:** Impossível registar um departamento sem formadores
- **Depois:** Departamentos existem independentemente dos formadores

5. Impacto no desempenho

O modelo normalizado requer mais JOINS nas consultas, mas elimina redundâncias e inconsistências. Esta é uma decisão consciente apropriada para sistemas transacionais.

6. Documentação é crucial

Cada decisão de design deve ser documentada com a justificação. Isto facilita manutenção futura e ajuda novos membros da equipa a compreender o modelo.

Glossário

Atomicidade

Propriedade de um valor que não pode ser dividido em componentes mais pequenos significativos no contexto do modelo de dados.

Chave candidata

Conjunto de um ou mais atributos que pode identificar unicamente cada registo numa tabela. Uma tabela pode ter várias chaves candidatas.

Chave composta

Chave primária formada por dois ou mais campos que, em conjunto, identificam unicamente cada registo.

Chave estrangeira

Campo numa tabela que referencia a chave primária de outra tabela, estabelecendo uma relação entre as duas.

Dependência funcional

Relação entre dois conjuntos de atributos onde os valores de um conjunto determinam unicamente os valores do outro. Notação: $X \rightarrow Y$ (X determina Y).

Dependência parcial

Situação onde um atributo não chave depende apenas de parte de uma chave primária composta.

Dependência transitiva

Situação onde um atributo não chave depende de outro atributo não chave, criando uma cadeia de dependências.

Desnormalização

Processo intencional de adicionar redundância a um modelo normalizado para melhorar o desempenho de consultas específicas.

Superchave

Conjunto de atributos que pode identificar unicamente cada registo. Pode conter atributos além dos estritamente necessários (ao contrário da chave candidata, que é minimal).

Questões frequentes (FAQ)

“

Se a normalização é tão importante, porque é que alguns sistemas usam estruturas desnormalizadas?

A normalização é essencial para sistemas transacionais (OLTP) onde a integridade e consistência dos dados são prioritárias. No entanto, em sistemas analíticos (OLAP) e data warehouses, a desnormalização intencional pode melhorar dramaticamente o desempenho das consultas. A decisão depende do tipo de sistema e dos requisitos específicos. O importante é que seja uma escolha consciente, não um erro de design.

”

“

É sempre necessário normalizar até BCNF?

Não. Para a maioria dos sistemas empresariais, a 3NF é suficiente. A BCNF resolve casos específicos e relativamente raros que envolvem múltiplas chaves candidatas sobrepostas. Se o vosso modelo está em 3NF e não identificam violações de BCNF, não há necessidade de complexificar.

”

“

Como decido quando usar uma chave primária simples (ID) vs. uma chave composta?

As chaves compostas são naturais quando modelamos relações muitos para muitos (como Inscricoes com ID_Formando e ID_Curso). No entanto, adicionar um ID_Inscricao simplifica operações futuras como atualizações e referências. A decisão deve equilibrar a naturalidade do modelo com a praticidade operacional.

”

“

Posso perder informação durante o processo de normalização?

Não, se o processo for executado corretamente. A normalização reorganiza os dados mas preserva toda a informação através de relações entre tabelas. Se conseguem reconstruir todos os dados originais através de JOINS, não perderam informação. Este princípio chama-se "decomposição sem perdas".

”

“

E se os requisitos de negócio mudarem após normalizar?

Um modelo bem normalizado é mais flexível a mudanças. Por exemplo, se a empresa decidir que cada curso pode ter múltiplos formadores, no nosso modelo bastaria criar uma tabela de associação Cursos_Formadores. Num modelo mal desenhado, esta mudança poderia exigir reestruturação completa.

”

“

Como lido com atributos que são "calculáveis" a partir de outros?

Atributos derivados (como "Total = Quantidade × Preço") normalmente não devem ser armazenados, pois violam a 3NF (dependem de outros atributos não chave). Devem ser calculados nas consultas. Exceção: Quando o cálculo é computacionalmente caro e os dados base raramente mudam, pode justificar-se armazenar o valor calculado por razões de desempenho.

”